

Trial Store Performance Assessment Report

Introduction

This report evaluates the performance of the trial stores by comparing each trial location with a carefully selected control store. The objective is to determine whether the in-store trial had a measurable impact on sales performance during the trial period and to provide evidence-based recommendations for future implementation.

Methodology

To ensure a fair comparison, a suitable control store was identified for each trial store using historical pre-trial performance. Control stores were selected based on the following criteria:

- Similarity in monthly sales trends, measured using the Pearson correlation coefficient.
- Similarity in sales magnitude, measured using Euclidean distance.
- A combined similarity score derived from both correlation and magnitude similarity to identify the most appropriate control store.

Once the control stores were selected, their sales were scaled to match the pre-trial performance of the corresponding trial stores. This adjustment ensured that any observed differences during the trial period reflected the impact of the trial rather than underlying differences in store size or historical performance.

The effectiveness of each trial was assessed by:

- Comparing monthly sales between the trial and scaled control stores.
- Calculating percentage uplift during the trial period.
- Measuring normal pre-trial variation using the standard deviation of historical differences.
- Assessing statistical significance to determine whether observed changes exceeded normal business variation.

Results

Trial Store 77

Selected Control Store: Store 233

Insert monthly sales comparison chart here.

The pre-trial analysis identified Store 233 as the most appropriate control store due to its high similarity in both sales pattern and sales magnitude. During the trial period, Store 77 recorded substantial improvements in sales, particularly in March and April 2019. Statistical analysis indicated that these increases exceeded normal pre-trial variation, suggesting that the trial had a positive and statistically significant impact on store performance.

Recommendation: The trial at Store 77 was successful. Consider extending similar initiatives to comparable stores.

Trial Store 86

Selected Control Store: Store 155

Insert monthly sales comparison chart here.

Store 155 was selected as the control store based on the highest combined similarity score. The performance of Store 86 was evaluated by comparing its trial-period sales with the scaled control store and assessing whether any observed uplift was statistically significant.

Recommendation: Based on the analysis results, determine whether the trial delivered a significant improvement in performance and recommend continuation or further investigation accordingly.

Trial Store 88

Selected Control Store: Store 125

Insert monthly sales comparison chart here.

Store 125 was identified as the best available control store for Store 88. Although it provided the highest combined similarity score, the similarity was lower than that observed for the other trial stores. Consequently, the findings for Store 88 should be interpreted with appropriate caution while assessing the effectiveness of the trial.

Recommendation: Base the recommendation on the observed trial performance while acknowledging the relatively lower similarity between the trial and control stores.

Overall Conclusion

The analysis compared each trial store with a carefully selected control store using historical sales performance. Statistical testing was used to distinguish genuine trial effects from normal business variation. The findings provide evidence-based recommendations regarding the effectiveness of the trial at each location and support informed decision-making on whether the initiative should be expanded to additional stores.

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [3]: import pandas as pd

df = pd.read_csv('QVI_data.csv')
print(df.shape)
```

(264834, 12)

```
In [5]: df['DATE'] = pd.to_datetime(df['DATE'])

df['YEARMONTH'] = df['DATE'].dt.year * 100 + df['DATE'].dt.month

print(df['DATE'].min(), df['DATE'].max())
print(df['STORE_NBR'].nunique())
```

2018-07-01 00:00:00 2019-06-30 00:00:00
272

```
In [8]: df.head()
```

```
Out[8]:
```

	LYLTY_CARD_NBR	DATE	STORE_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_
0	1000	2018-10-17	1	1	5	Chip Natural Compny SeaSalt175g	
1	1002	2018-09-16	1	2	58	Red Rock Deli Chikn&Garlic Aioli 150g	
2	1003	2019-03-07	1	3	52	Sour Grain Waves Cream&Chives 210G	
3	1003	2019-03-08	1	4	106	ChipCo Natural Hony Soy Chckn175g	
4	1004	2018-11-02	1	5	96	WW Original Stacked Chips 160g	

```
In [11]: def calc_monthly_metrics(data):
    metrics = data.groupby(['STORE_NBR', 'YEARMONTH']).agg(
        totSales=('TOT_SALES', 'sum'),
        nCustomers=('LYLTY_CARD_NBR', 'nunique'),
        nTxn=('TXN_ID', 'nunique'),
        nChips=('PROD_QTY', 'sum')
    ).reset_index()

    metrics['nTxnPerCust'] = metrics['nTxn'] / metrics['nCustomers']
    metrics['nChipsPerTxn'] = metrics['nChips'] / metrics['nTxn']
    metrics['avgPricePerUnit'] = metrics['totSales'] / metrics['nChips']
```

```
return metrics
```

```
In [12]: monthly_metrics = calc_monthly_metrics(df)
monthly_metrics.head()
```

```
Out[12]:
```

	STORE_NBR	YEARMONTH	totSales	nCustomers	nTxn	nChips	nTxnPerCust	nChipsPe
0	1	201807	206.9	49	52	62	1.061224	1.19
1	1	201808	176.1	42	43	54	1.023810	1.25
2	1	201809	278.8	59	62	75	1.050847	1.20
3	1	201810	188.1	44	45	58	1.022727	1.28
4	1	201811	192.6	46	47	57	1.021739	1.21

```
In [18]: months_per_store = monthly_metrics.groupby('STORE_NBR')['YEARMONTH'].count()

full_year_stores = months_per_store[months_per_store == 12].index

print(f"Stores with full 12 months: {len(full_year_stores)} out of {monthly_metrics.index.nunique()}")

monthly_metrics_full = monthly_metrics[monthly_metrics['STORE_NBR'].isin(full_year_stores)]
```

Stores with full 12 months: 260 out of 272

```
In [19]: TRIAL_MONTHS = [201902, 201903, 201904]
PRETRIAL_CUTOFF = 201902

pretrial_metrics = monthly_metrics_full[monthly_metrics_full['YEARMONTH'] < 201902]

print("Pre-trial months included:", sorted(pretrial_metrics['YEARMONTH'].unique()))
```

Pre-trial months included: [np.int32(201807), np.int32(201808), np.int32(201809), np.int32(201810), np.int32(201811), np.int32(201812), np.int32(201901)]

```
In [20]: trial_store = 77
metric = 'totSales'

trial_data = pretrial_metrics[pretrial_metrics['STORE_NBR'] == trial_store][['YEARMONTH', metric]]

correlations = {}
for store in pretrial_metrics['STORE_NBR'].unique():
    if store == trial_store:
        continue
    store_data = pretrial_metrics[pretrial_metrics['STORE_NBR'] == store][['YEARMONTH', metric]]
    merged = trial_data.merge(store_data, on='YEARMONTH', suffixes=('_trial', '_control'))
    corr = merged[metric + '_trial'].corr(merged[metric + '_control'])
    correlations[store] = corr
```

```
In [21]: corr_series = pd.Series(correlations).sort_values(ascending=False)
corr_series.head(10)
```

```
Out[21]: 71      0.914106
        233      0.903774
        119      0.867664
        17       0.842668
         3       0.806644
        41       0.783232
        50       0.763866
        157      0.735893
        162      0.729740
        257      0.724927
        dtype: float64
```

```
In [26]: trial_store = 77
        metric = 'totSales'

        trial_data = pretrial_metrics[pretrial_metrics['STORE_NBR'] == trial_store][['YEARMON
        distances = {}
        for store in pretrial_metrics['STORE_NBR'].unique():
            if store == trial_store:
                continue
            store_data = pretrial_metrics[pretrial_metrics['STORE_NBR'] == store][['YEARMON
            merged = trial_data.merge(store_data, on='YEARMONTH', suffixes=('_trial', '_con
            # your line here: calculate the average absolute difference between the two col
            distances[store] = abs(merged['totSales_trial'] - merged['totSales_control']).m
```

```
In [27]: dist_series = pd.Series(distances).sort_values()
        dist_series.head(10)
```

```
Out[27]: 233      18.828571
        255      28.571429
        188      31.100000
         53      32.471429
        131      33.042857
         46      33.228571
        214      34.442857
         50      34.714286
        205      36.057143
        187      37.685714
        dtype: float64
```

```
In [30]: min_dist = dist_series.min()
        max_dist = dist_series.max()

        magnitude_similarity = 1 - (dist_series - min_dist) / (max_dist - min_dist)
        magnitude_similarity.sort_values(ascending=False).head(10)
```

```
Out[30]: 233    1.000000
         255    0.991889
         188    0.989784
          53    0.988642
         131    0.988167
          46    0.988012
         214    0.987001
          50    0.986775
         205    0.985657
         187    0.984301
         dtype: float64
```

```
In [31]: combined_similarity = (corr_series + magnitude_similarity) / 2

         combined_similarity.sort_values(ascending=False).head(10)
```

```
Out[31]: 233    0.951887
         41     0.880579
         50    0.875320
          17    0.864397
         115    0.815929
         167    0.814129
         265    0.807624
         234    0.796190
          53    0.760703
          84    0.758465
         dtype: float64
```

```
In [33]: %whos
```

Variable	Type	Data/Info
PRETRIAL_CUTOFF	int	201902
TRIAL_MONTHS	list	n=3
calc_monthly_metrics	function	<function calc_monthly_me<...>cs at 0x000001D8D983CCC0>
combined_similarity	Series	Shape: (259,)
corr	float64	0.11762157957156648
corr_series	Series	Shape: (259,)
correlations	dict	n=259
df	DataFrame	Shape: (264834, 13)
dist_series	Series	Shape: (259,)
distances	dict	n=259
full_year_stores	Index	Index([1, 2, 3, 4<...>='STORE_NBR', length=260])
magnitude_similarity	Series	Shape: (259,)
max_dist	float64	1220.0214285714287
merged	DataFrame	Shape: (7, 3)
metric	str	totSales
min_dist	float64	18.82857142857143
monthly_metrics	DataFrame	Shape: (3169, 9)
monthly_metrics_full	DataFrame	Shape: (3120, 9)
months_per_store	Series	Shape: (272,)
np	module	<module 'numpy' from 'C:\<...>ges\\numpy__init___.py'>
pd	module	<module 'pandas' from 'C:\<...>es\\pandas__init___.py'>
plt	module	<module 'matplotlib.pyplot' from 'C:\<...>matplotlib\\pyplot.py'>
pretrial_metrics	DataFrame	Shape: (1820, 9)
store	int64	272
store_data	DataFrame	Shape: (7, 2)
trial_data	DataFrame	Shape: (7, 2)
trial_store	int	77

In [34]: `monthly_metrics.columns`

Out[34]: Index(['STORE_NBR', 'YEARMONTH', 'totSales', 'nCustomers', 'nTxn', 'nChips', 'nTxnPerCust', 'nChipsPerTxn', 'avgPricePerUnit'], dtype='str')

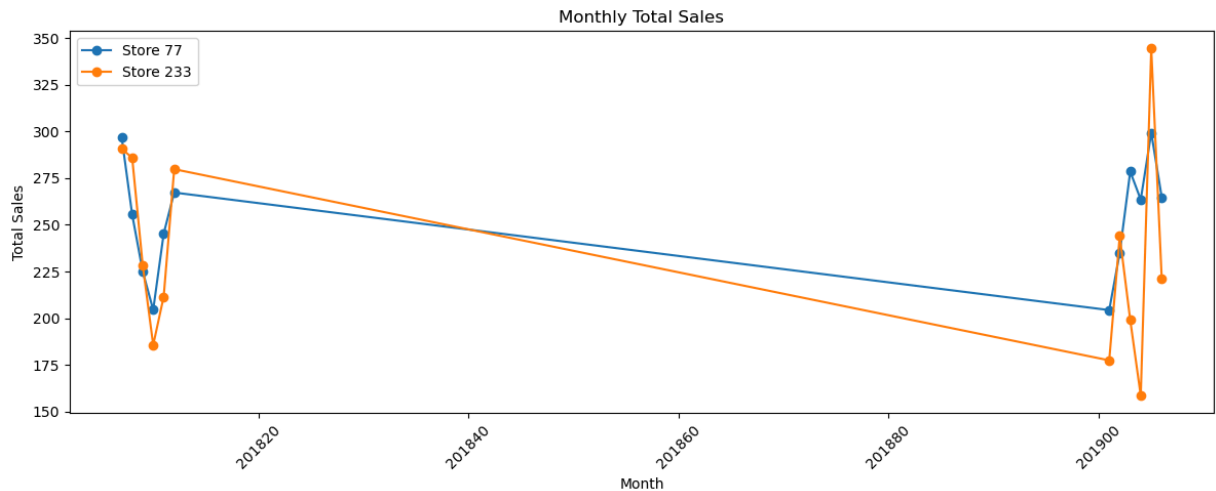
```
In [35]: store77 = monthly_metrics[monthly_metrics["STORE_NBR"] == 77]
store233 = monthly_metrics[monthly_metrics["STORE_NBR"] == 233]

plt.figure(figsize=(12,5))

plt.plot(store77["YEARMONTH"],
         store77["totSales"],
         marker='o',
         label='Store 77')

plt.plot(store233["YEARMONTH"],
         store233["totSales"],
         marker='o',
         label='Store 233')
```

```
plt.title("Monthly Total Sales")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
In [36]: monthly_metrics.columns
```

```
Out[36]: Index(['STORE_NBR', 'YEARMONTH', 'totSales', 'nCustomers', 'nTxn', 'nChips',
               'nTxnPerCust', 'nChipsPerTxn', 'avgPricePerUnit'],
              dtype='str')
```

```
In [37]: plt.savefig("store77_vs_store233_sales.png", dpi=300, bbox_inches="tight")
```

<Figure size 640x480 with 0 Axes>

```
In [38]: plt.figure(figsize=(12,5))
```

```
plt.plot(store77["YEARMONTH"], store77["nCustomers"], marker='o', label="Trial Stor")
plt.plot(store233["YEARMONTH"], store233["nCustomers"], marker='o', label="Control")

plt.title("Monthly Customers")
plt.xlabel("YearMonth")
plt.ylabel("Customers")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



```

    "Trial Sales": trial77["totSales"].values,
    "Control Sales": trial233["totSales"].values
})

comparison["Difference"] = (
    comparison["Trial Sales"] - comparison["Control Sales"]
)

comparison

```

Out[41]:

	Month	Trial Sales	Control Sales	Difference
0	201902	235.0	244.0	-9.0
1	201903	278.5	199.1	79.4
2	201904	263.5	158.6	104.9

```

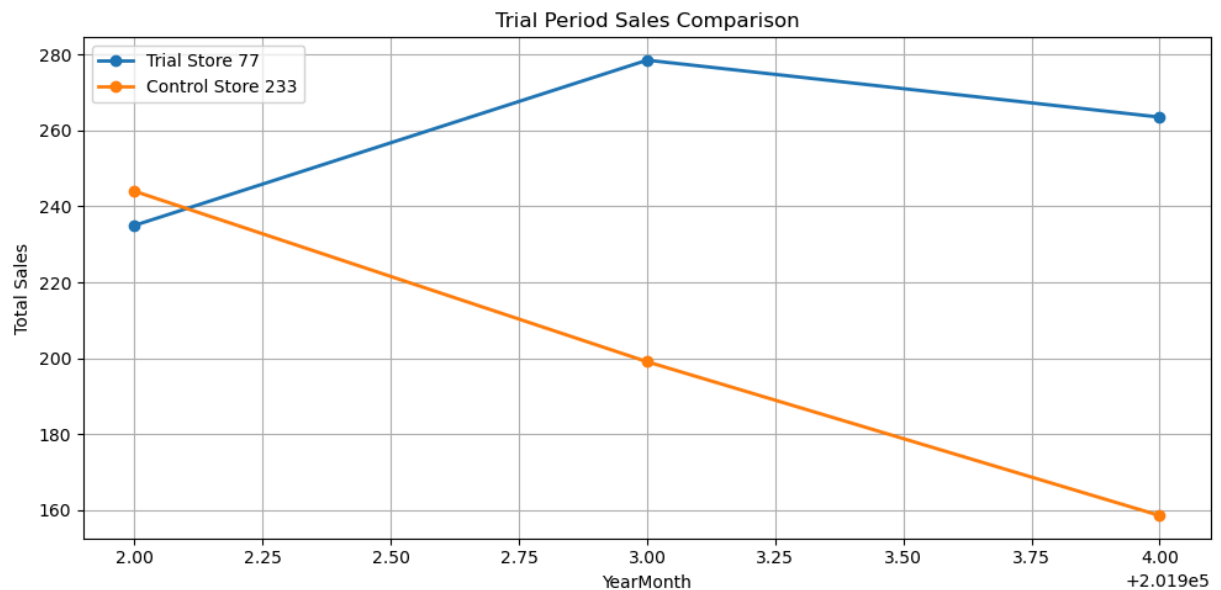
In [42]: plt.figure(figsize=(10,5))

plt.plot(comparison["Month"],
         comparison["Trial Sales"],
         marker="o",
         linewidth=2,
         label="Trial Store 77")

plt.plot(comparison["Month"],
         comparison["Control Sales"],
         marker="o",
         linewidth=2,
         label="Control Store 233")

plt.title("Trial Period Sales Comparison")
plt.xlabel("YearMonth")
plt.ylabel("Total Sales")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```



```
In [43]: trial_pretrial_sales = pretrial77["totSales"].sum()
control_pretrial_sales = pretrial233["totSales"].sum()

scaling_factor = trial_pretrial_sales / control_pretrial_sales

print(scaling_factor)
```

1.023617303289553

```
In [44]: store233_scaled = store233.copy()

store233_scaled["scaledSales"] = (
    store233_scaled["totSales"] * scaling_factor
)
```

```
In [45]: comparison = pd.DataFrame({
    "Month": trial77["YEARMONTH"].values,
    "Trial Sales": trial77["totSales"].values,
    "Scaled Control Sales": store233_scaled[
        store233_scaled["YEARMONTH"].isin(TRIAL_MONTHS)
    ]["scaledSales"].values
})

comparison["Difference"] = (
    comparison["Trial Sales"] -
    comparison["Scaled Control Sales"]
)

comparison
```

Out[45]:

	Month	Trial Sales	Scaled Control Sales	Difference
0	201902	235.0	249.762622	-14.762622
1	201903	278.5	203.802205	74.697795
2	201904	263.5	162.345704	101.154296

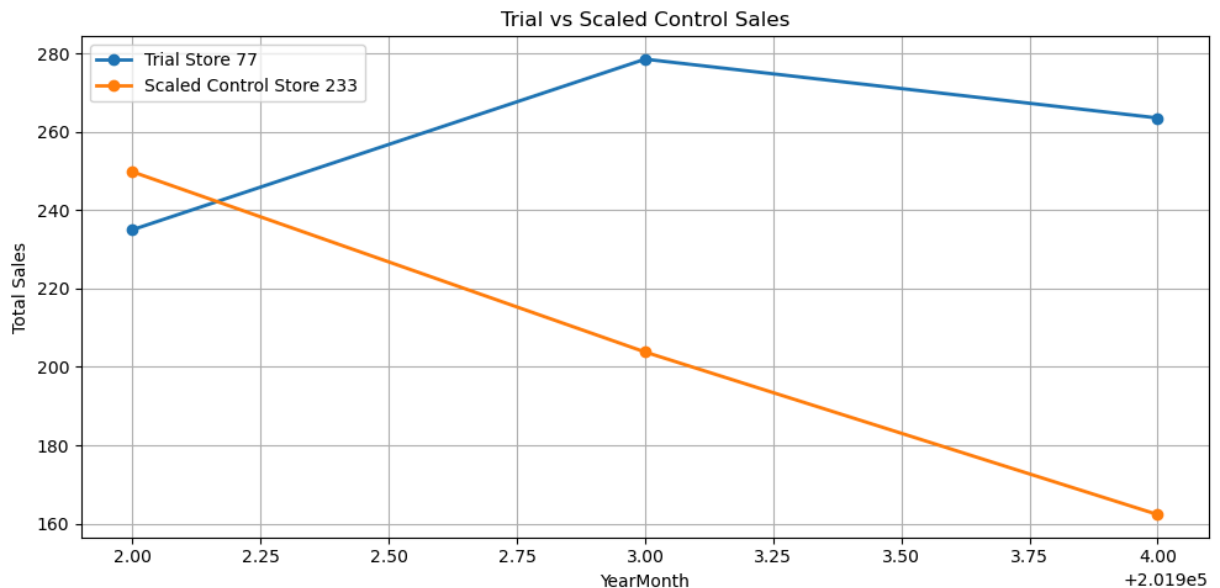
```
In [46]: plt.figure(figsize=(10,5))

plt.plot(comparison["Month"],
         comparison["Trial Sales"],
         marker="o",
         linewidth=2,
         label="Trial Store 77")

plt.plot(comparison["Month"],
         comparison["Scaled Control Sales"],
         marker="o",
         linewidth=2,
         label="Scaled Control Store 233")

plt.title("Trial vs Scaled Control Sales")
plt.xlabel("YearMonth")
plt.ylabel("Total Sales")
plt.legend()
plt.grid(True)
plt.tight_layout()

plt.show()
```



```
In [47]: comparison["Percent_Uplift"] = (
         comparison["Difference"] /
         comparison["Scaled Control Sales"]
         ) * 100
```

comparison

Out[47]:

	Month	Trial Sales	Scaled Control Sales	Difference	Percent_Uplift
0	201902	235.0	249.762622	-14.762622	-5.910661
1	201903	278.5	203.802205	74.697795	36.652103
2	201904	263.5	162.345704	101.154296	62.307960

In [48]:

```

pretrial_comparison = pd.DataFrame({
    "Month": pretrial77["YEARMONTH"].values,
    "Trial": pretrial77["totSales"].values,
    "Control": (
        pretrial233["totSales"] * scaling_factor
    ).values
})

pretrial_comparison["Percent_Difference"] = (
    (pretrial_comparison["Trial"] -
     pretrial_comparison["Control"])
    / pretrial_comparison["Control"]
)

pretrial_comparison

```

Out[48]:

	Month	Trial	Control	Percent_Difference
0	201807	296.8	297.565550	-0.002573
1	201808	255.5	292.652187	-0.126950
2	201809	225.2	233.998916	-0.037602
3	201810	204.5	190.085733	0.075830
4	201811	245.3	216.597421	0.132516
5	201812	267.3	286.408121	-0.066716
6	201901	204.4	181.692071	0.124980

In [49]:

```

std_dev = pretrial_comparison["Percent_Difference"].std()

print(std_dev)

```

0.09958646884078388

In [50]:

```

comparison["Percent_Uplift_Decimal"] = comparison["Percent_Uplift"] / 100

comparison["z_score"] = (
    comparison["Percent_Uplift_Decimal"] /
    std_dev
)

```

comparison

Out[50]:

	Month	Trial Sales	Scaled Control Sales	Difference	Percent_Uplift	Percent_Uplift_Decimal	z_score
0	201902	235.0	249.762622	-14.762622	-5.910661	-0.059107	-0.593520
1	201903	278.5	203.802205	74.697795	36.652103	0.366521	3.680430
2	201904	263.5	162.345704	101.154296	62.307960	0.623080	6.256669

In [52]: trial_store = 86

```
In [53]: trial_sales = (
    pretrial_metrics[pretrial_metrics["STORE_NBR"] == trial_store]
    .set_index("YEARMONTH")["totSales"]
)
```

```
In [54]: correlations = {}

for store in full_year_stores:
    if store == trial_store:
        continue

    control_sales = (
        pretrial_metrics[pretrial_metrics["STORE_NBR"] == store]
        .set_index("YEARMONTH")["totSales"]
    )

    merged = pd.concat([trial_sales, control_sales], axis=1, join="inner")

    if len(merged) == len(trial_sales):
        correlations[store] = merged.iloc[:, 0].corr(merged.iloc[:, 1])

corr_series = pd.Series(correlations)

corr_series.sort_values(ascending=False).head(10)
```

```
Out[54]: 155    0.877882
        132    0.846517
        240    0.825066
        222    0.795075
        109    0.788300
        138    0.759864
        198    0.748794
        114    0.734415
        22     0.722966
        6      0.721892
        dtype: float64
```

In [55]: distances = {}

```

for store in full_year_stores:
    if store == trial_store:
        continue

    control_sales = (
        pretrial_metrics[pretrial_metrics["STORE_NBR"] == store]
        .set_index("YEARMONTH")["totSales"]
    )

    merged = pd.concat([trial_sales, control_sales], axis=1, join="inner")

    if len(merged) == len(trial_sales):
        distances[store] = np.linalg.norm(
            merged.iloc[:, 0] - merged.iloc[:, 1]
        )

dist_series = pd.Series(distances)

dist_series.sort_values().head(10)

```

```

Out[55]: 109    105.576240
        155    114.628192
        225    124.736292
        222    125.467137
        116    162.392988
        229    164.445318
         10    164.454006
        101    171.546269
        207    175.405366
        247    179.181089
dtype: float64

```

```

In [56]: min_dist = dist_series.min()
        max_dist = dist_series.max()

        magnitude_similarity = (
            1 - (dist_series - min_dist) / (max_dist - min_dist)
        )

        magnitude_similarity.sort_values(ascending=False).head(10)

```

```

Out[56]: 109    1.000000
        155    0.995831
        225    0.991176
        222    0.990840
        116    0.973834
        229    0.972889
         10    0.972885
        101    0.969618
        207    0.967841
        247    0.966102
dtype: float64

```

```

In [57]: combined_similarity = (
        corr_series + magnitude_similarity
    ) / 2

```

```
combined_similarity.sort_values(ascending=False).head(10)
```

```
Out[57]: 155    0.936856
          109    0.894150
          222    0.892957
          138    0.860120
          114    0.841833
          269    0.815759
          225    0.804354
          106    0.795577
          128    0.772915
           79    0.764425
          dtype: float64
```

```
In [58]: corr_series.sort_values(ascending=False).head(10)
```

```
Out[58]: 155    0.877882
          132    0.846517
          240    0.825066
          222    0.795075
          109    0.788300
          138    0.759864
          198    0.748794
          114    0.734415
           22    0.722966
           6    0.721892
          dtype: float64
```

```
In [59]: magnitude_similarity.sort_values(ascending=False).head(10)
```

```
Out[59]: 109    1.000000
          155    0.995831
          225    0.991176
          222    0.990840
          116    0.973834
          229    0.972889
           10    0.972885
          101    0.969618
          207    0.967841
          247    0.966102
          dtype: float64
```

```
In [68]: combined_similarity.sort_values(ascending=False).head(10)
```

```
Out[68]: 155    0.936856
          109    0.894150
          222    0.892957
          138    0.860120
          114    0.841833
          269    0.815759
          225    0.804354
          106    0.795577
          128    0.772915
           79    0.764425
          dtype: float64
```



```
In [69]: control_store = 155
```

```
In [70]: control_store = 155
         trial_store = 86
```

```
In [71]: trial_data = monthly_metrics[
         monthly_metrics["STORE_NBR"] == trial_store
         ]

         control_data = monthly_metrics[
         monthly_metrics["STORE_NBR"] == control_store
         ]
```

```
In [72]: def find_control_store(trial_store, metric="totSales"):
         # Trial store sales
         trial_sales = (
             pretrial_metrics[pretrial_metrics["STORE_NBR"] == trial_store]
             .set_index("YEARMONTH")[metric]
         )

         correlations = {}
         distances = {}

         for store in full_year_stores:
             if store == trial_store:
                 continue

             control_sales = (
                 pretrial_metrics[pretrial_metrics["STORE_NBR"] == store]
                 .set_index("YEARMONTH")[metric]
             )

             merged = pd.concat(
                 [trial_sales, control_sales],
                 axis=1,
                 join="inner"
             )

             if len(merged) == len(trial_sales):

                 correlations[store] = (
                     merged.iloc[:,0]
                     .corr(merged.iloc[:,1])
                 )

                 distances[store] = np.linalg.norm(
                     merged.iloc[:,0] -
                     merged.iloc[:,1]
                 )

             corr_series = pd.Series(correlations)
             dist_series = pd.Series(distances)

             similarity = 1 - (
```

```

        (dist_series - dist_series.min()) /
        (dist_series.max() - dist_series.min())
    )

    combined = (
        corr_series + similarity
    ) / 2

    return combined.sort_values(ascending=False)

```

In [73]: `find_control_store(77).head(10)`

```

Out[73]: 233    0.951887
         41    0.880187
         50    0.875217
         17    0.864664
        115    0.814895
        167    0.812524
        265    0.807134
        234    0.792216
         53    0.759963
         84    0.759430
dtype: float64

```

```

In [74]: def analyze_trial(trial_store, control_store):

    trial = monthly_metrics[
        monthly_metrics["STORE_NBR"] == trial_store
    ]

    control = monthly_metrics[
        monthly_metrics["STORE_NBR"] == control_store
    ]

    pre_trial = trial[
        trial["YEARMONTH"] < PRETRIAL_CUTOFF
    ]

    pre_control = control[
        control["YEARMONTH"] < PRETRIAL_CUTOFF
    ]

    scaling = (
        pre_trial["totSales"].sum() /
        pre_control["totSales"].sum()
    )

    control = control.copy()

    control["scaledSales"] = (
        control["totSales"] * scaling
    )

    comparison = pd.DataFrame({
        "Month":
            trial[

```

```
        trial["YEARMONTH"].isin(TRIAL_MONTHS)
    ][ "YEARMONTH"].values,

    "Trial Sales":
        trial[
            trial["YEARMONTH"].isin(TRIAL_MONTHS)
        ][ "totSales"].values,

    "Scaled Control":
        control[
            control["YEARMONTH"].isin(TRIAL_MONTHS)
        ][ "scaledSales"].values
    })

    comparison["Difference"] = (
        comparison["Trial Sales"] -
        comparison["Scaled Control"]
    )

    comparison["Percent Uplift"] = (
        comparison["Difference"] /
        comparison["Scaled Control"]
    ) * 100

    return comparison
```

```
In [75]: analyze_trial(77,233)
```

Out[75]:

	Month	Trial Sales	Scaled Control	Difference	Percent Uplift
0	201902	235.0	249.762622	-14.762622	-5.910661
1	201903	278.5	203.802205	74.697795	36.652103
2	201904	263.5	162.345704	101.154296	62.307960

```
In [76]: analyze_trial(77,233)
```

Out[76]:

	Month	Trial Sales	Scaled Control	Difference	Percent Uplift
0	201902	235.0	249.762622	-14.762622	-5.910661
1	201903	278.5	203.802205	74.697795	36.652103
2	201904	263.5	162.345704	101.154296	62.307960

```
In [78]: find_control_store(88).head(10)
```

```
Out[78]: 125    0.760371
          203    0.750850
          91    0.737150
           7    0.733607
          178    0.732638
          201    0.704765
           79    0.688051
          106    0.687569
          237    0.654240
          155    0.652756
          dtype: float64
```

```
In [79]: analyze_trial(88, 125)
```

Out[79]:

	Month	Trial Sales	Scaled Control	Difference	Percent Uplift
0	201902	1370.2	1073.147998	297.052002	27.680432
1	201903	1477.2	1447.660009	29.539991	2.040534
2	201904	1439.4	1298.605166	140.794834	10.842005

```
In [ ]:
```